WEEK 05 · RAZOR PAGES FORM FLOW

# 讓表單
# 真的收到資料

小安的活動網站已經好看了，但同學按下「送出」後，資料要去哪裡？今天把畫面接上 ASP.NET 後端，完成第一條可驗證的報名流程。

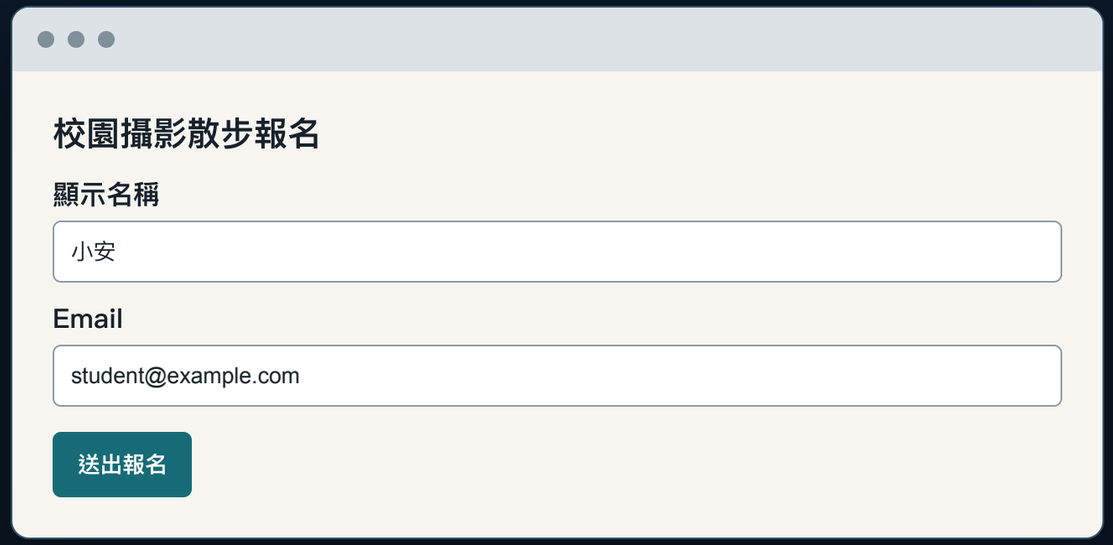
下載學生任務包 ↓

下載 PDF ↓



EXPECTED RESULT · 先看終點

# 今天的終點不是「有表單」，而是錯的擋住、對的收到。




開啟可操作 Demo ↗

ASP.NET 回應

## 報名資料已收到

參加者：小安
Email：student@example.com
場次：週三晚上

**限制：**今天還沒有寫入資料庫。

MISSION MAP

# 三節課，只完成一條資料路線。

1

### 做出可填表單

認識 **form**、**label**、**input** 與 Tag Helper，完成可操作的表單畫面。

2

### 讓後端收到

用 **BindProperty** 與 **OnPost**，看見輸入值進入 PageModel。

3

### 擋住錯誤資料

用 **DataAnnotations** 與 **ModelState**，測試空白、錯誤與成功三條路。

今天不接資料庫。先把「送出 → 接收 → 驗證 → 回應」做懂、做穩。

THE WHOLE JOURNEY

# 一筆資料，會經過五個站。

1 瀏覽器
填寫欄位
→
2 POST
送出請求
→
3 Binding
放進物件
→
4 Validation
檢查規則
→
5 Razor Page
顯示結果

下一站
從頭看

使用者先在瀏覽器填入課堂假資料；現在資料還沒有離開畫面。

PROJECT MAP · 先認路，不用背

# 今天只要知道：畫面、C#、CSS 在哪裡。

```
Week05TeacherDemo/
├─ Pages/
│  ├─ Index.cshtml
│  └─ Index.cshtml.cs
└─ wwwroot/
   └─ css/site.css
```

**今天只在三處移動**

改畫面：`Pages/*.cshtml`

處理資料：`Pages/*.cshtml.cs`

改樣式：`wwwroot/css/`

**先認路**：`Index.cshtml` 與 `Index.cshtml.cs` 是同一個 Razor Page 的合作夥伴。其他檔案先知道存在，未來用到再介紹。

POINT 01 · GET VS POST

# GET 是拿報名頁，
# POST 是交出欄位資料。

第一次開啟首頁，瀏覽器向伺服器拿 HTML；填完按下送出，才把 form 內的資料交給後端。

### GET / 拿頁面

```
瀏覽器 ── GET / ──→ 伺服器
瀏覽器 ←── 表單 HTML ──
```

對應 Razor Pages 的 **OnGet**。

### POST / 交資料

```
瀏覽器 ── POST / ─→ 伺服器
          名稱、Email、場次
```

對應今天要完成的 **OnPost**。

POINT 02 · FORM BOUNDARY

# form 像一個資料信封，只有信封裡的欄位會一起送出。

```
<form method="post">
  <label>顯示名稱</label>
  <input ...>

  <button type="submit">
    送出報名
  </button>
</form>
```

## 先預測

如果 input 放在 </form> 後面，按下送出時，它會一起送嗎？

```
</form>
<input name="note"> <!-- Form 表單外 -->
```

不會

會

先選答案，再用「信封範圍」解釋。

method="post" 描述這次送出的方式

POINT 03 · LABEL BEFORE STYLE

# label 不是裝飾，
# 它把「問題」和「輸入位置」連起來。




```
<label for="nameDemo">
  顯示名稱
</label>
<input id="nameDemo">
```

Placeholder 只是範例，輸入後會消失；不能取代 label。

POINT 04 · TAG HELPER

# asp-for 告訴 Razor：這個欄位對應哪一個 C# 屬性。

```
Pages/
├─ Index.cshtml
└─ Index.cshtml.cs
```

```
<label asp-for="Input.DisplayName"></label>
<input asp-for="Input.DisplayName"
       class="form-control">
```

**畫面欄位**

Input.DisplayName

↔

**C# 屬性**

Input.DisplayName

兩邊名稱一致，才能沿著同一條資料線走。

暫停講解 · 學生實作

PAUSE & PRACTICE 01

# 先完成可操作的表單畫面。

15 分鐘

查看預期成品 Demo ↗

## 學生現在完成

1. 備份 Week04 的首頁與 PageModel。
2. 加入 form、名稱、Email、場次及備註。
3. 每個欄位使用 label 與 asp-for。
4. 保留自己的配色與 Bootstrap 卡片。

## 完成條件

- Tab 能依序進入每個欄位。
- 點 label 會對焦正確欄位。
- 390px 沒有水平捲動。
- 按鈕位於 form 內；此時尚未要求成功送出。

**老師巡視：**不要把完整 html/head/body 貼進 Razor 首頁。

**第 8 分鐘：**還沒完成者先只保留名稱、Email、場次三欄。

**快學生：**改善 label 與輸入框間距，不新增功能。

☐ 我已在自己的 localhost 實際操作表單欄位。

時間到：抽 2 位同學指出 form 範圍與 asp-for 對應

CHECKPOINT 01

# 如果畫面不對，先查三個地方。

1

### Razor 頂端

`@page`、`@model` 是否仍存在？不要貼第二組 html/head/body。

2

### form 邊界

欄位與 submit 按鈕是否都包在同一個 form 裡？

3

### 屬性名稱

`asp-for` 是否真的指向 PageModel 會有的屬性？

**此時按下送出出錯是合理的。**第一輪只完成表單外觀；下一輪才建立接收資料的 PageModel。

POINT 05 · PAGE MODEL

# Index.cshtml 負責呈現，Index.cshtml.cs 負責處理。

同一個 Razor Page 有兩個合作檔案：一個面對使用者，一個接住請求與資料。

```
Pages/
├─ Index.cshtml
│   └─ 表單、欄位、錯誤、結果
│
└─ Index.cshtml.cs
    └─ 屬性、OnGet、OnPost、驗證
```

前端與後端分工，但共同完成一頁

POINT 06 · INPUT MODEL

# 四個欄位，不要散成四條線；用一個 Input 物件收在一起。

```
public class RegistrationInput
{
    public string DisplayName { get; set; }
    public string Email { get; set; }
    public string Session { get; set; }
    public string Note { get; set; }
}
```

**Input**

DisplayName

**Input**

Email

**Input**

Session

**Input**

Note

同一個用途的輸入，組成一個模型

POINT 07 · MODEL BINDING

# [BindProperty] 讓 ASP.NET 把 POST 欄位放進 Input。

```
[BindProperty]
public RegistrationInput Input { get; set; }
    = new RegistrationInput();
```

### Binding 做了什麼？

1. 找出送來的欄位名稱。
2. 比對 Input.DisplayName 等屬性。
3. 把文字轉成屬性需要的型別。
4. 記錄轉換與驗證錯誤。

它不是資料庫。Binding 只把這次請求帶來的值放進 C# 物件。

POINT 08 · HANDLER

# 表單用 POST 送出時，Razor Pages 會尋找 OnPost()。

```
public bool Submitted { get; set; }

public void OnPost()
{
    Submitted = true;
}
```

### 和 OnGet 的分工

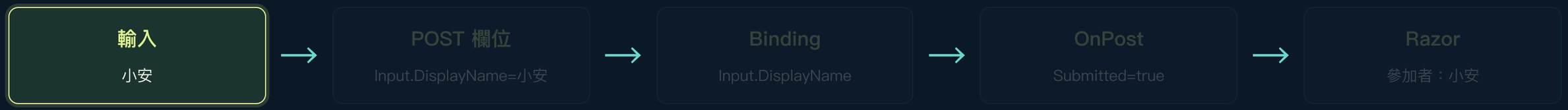
OnGet：第一次開啟頁面。

OnPost：使用者送出表單。

現在還沒有檢查內容。空白資料也會被當成成功；第三節才加上驗證閘門。

LIVE TRACE

# 「小安」如何走到成功摘要？




顯示下一步　重新追蹤

第一站：使用者在瀏覽器輸入「小安」。

每一站都可觀察，除錯才有方向

POINT 09 · RESPONSE

# 成功摘要不是另一個神祕頁面，是 Razor 根據狀態決定要不要顯示。

```
@if (Model.Submitted)
{
    <section class="alert alert-success">
      報名資料已收到
      參加者：@Model.Input.DisplayName
    </section>
}
```

### 現在能證明什麼？

後端收到這次 POST，並把 Input 的值放回 HTML 回應。

### 還不能證明什麼？

不能證明資料已存入 SQL Server，也不能證明重新啟動後還找得到。

正確描述系統現在做到哪裡

暫停講解 · 學生實作

PAUSE & PRACTICE 02

# 讓畫面資料進入 PageModel。

20 分鐘

查看預期成品 Demo ↗

## 學生現在完成

1. 建立 RegistrationInput。
2. 加入 Input 的 BindProperty。
3. 建立 OnPost 與 Submitted。
4. 送出後顯示名稱、Email、場次摘要。

## 完成條件

- 送出「小安」後，摘要也顯示「小安」。
- 換成另一個名稱，摘要跟著變。
- 能指出 cshtml 與 cshtml.cs 各自負責什麼。
- 能說明目前沒有資料庫保存。

**老師巡視：**先看錯誤訊息與對應行，不讓 AI 重寫整頁。

**第 10 分鐘：**卡住者只保留名稱、Email、場次三欄。

**快學生：**改善成功摘要排版，不增加新欄位。

- [ ] 我已用兩組不同假資料完成 POST 與摘要驗證。

時間到：請一位同學沿五站路線解釋資料

CHECKPOINT 02

# 送出沒有結果時，沿資料路線查，不要直接重裝或重寫。

1 form
method="post" ？
→
2 asp-for
名稱一致？
→
3 BindProperty
Input 可接收？
→
4 OnPost
有執行？
→
5 Razor
條件成立？

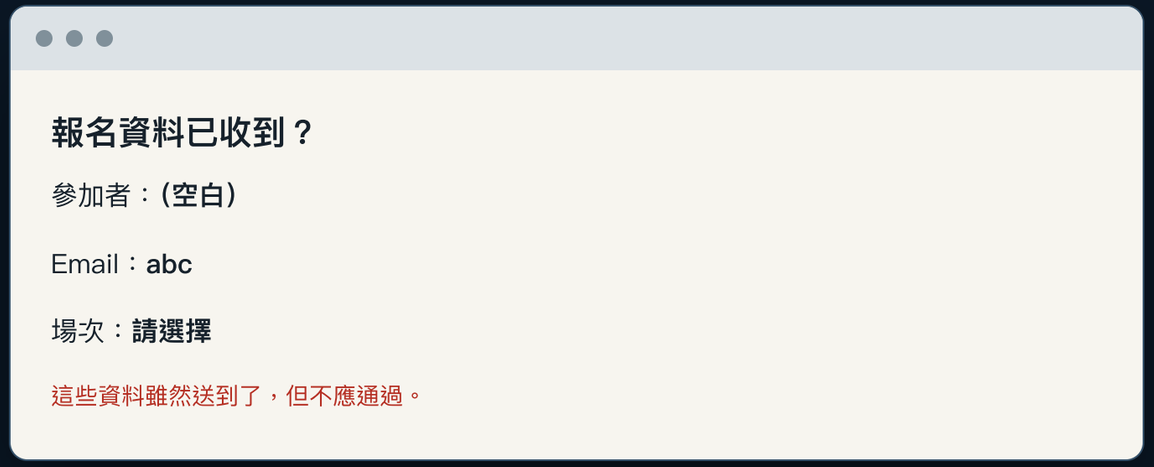
> 除錯問題要具體：「我輸入____，按下____，預期____，實際看到____。」

現象越具體，修正範圍越小

POINT 10 · RECEIVED ≠ VALID

# 後端收到資料，
# 不代表資料合理。

空白名字、錯誤 Email、未選場次都能被傳送。驗證的工作，是在成功之前設一道明確閘門。



POINT 11 · RULES BESIDE DATA

# 每個驗證標記，只回答一個問題。

```
[Required(ErrorMessage = "請輸入顯示名稱")]
[StringLength(20, ErrorMessage = "最多 20 字")]
[Display(Name = "顯示名稱")]
public string DisplayName { get; set; }
```

### Required

可以空白嗎？

### StringLength

最多可以多長？

### Display

畫面上要叫什麼名字？

一條規則，一個可測試問題

POINT 12 · VALIDATION GATE

# 先問 ModelState.IsValid，通過才打開成功區。

```
public void OnPost()
{
    if (!ModelState.IsValid)
    {
        return;
    }

    Submitted = true;
}
```

**規則未通過**

留在表單，顯示錯誤

**全部通過**

顯示成功摘要

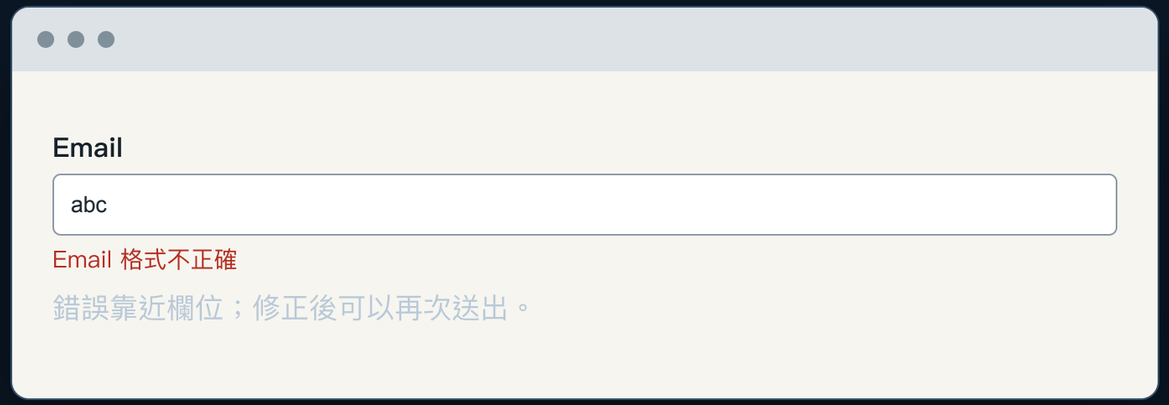
如果先設定 Submitted，再檢查 ModelState，空白資料也可能出現成功訊息。

成功訊息必須在驗證之後

POINT 13 · SHOW THE REPAIR PATH

# 驗證不是只說「失敗」，要讓使用者知道哪裡需要修。

```
<label asp-for="Input.Email"></label>
<input asp-for="Input.Email"
       class="form-control">
<span asp-validation-for="Input.Email"
      class="text-danger"></span>
```




錯誤訊息本身也是介面設計

POINT 14 · TEST BEHAVIOR

# 只測成功一次，不能證明驗證有效。

| 測試路線 | 輸入 | 應該看到 | 操作 |
| --- | --- | --- | --- |
| 空白 | 所有必填欄空白 | 必填錯誤；沒有成功摘要 | 模擬 |
| 錯誤格式 | Email = abc | Email 格式錯誤 | 模擬 |
| 正確 | 課堂假資料 | 成功摘要與輸入一致 | 模擬 |

選一條路，觀察預期行為。之後要在自己的 localhost 重做。

POINT 15 · AI DEBUG CONTRACT

# 不要請 AI 重寫整頁，給它一個可重現的小問題。

## ✕ 太模糊

```
表單壞了，幫我全部修好。
```

AI 不知道輸入、操作、預期與實際結果。

## ✓ 可驗證

```
Razor Pages 表單使用 POST。
輸入 Email=abc 時仍顯示成功。
以下只貼 OnPost 與 Email 屬性。
請先指出原因，再給最小修改與三組測試。
```

**讀** 理解建議 → **比** 查看差異 → **跑** 啟動專案 → **測** 三條路 → **追問** 描述現象

暫停講解 · 學生實作

PAUSE & PRACTICE 03

# 讓錯的停下，讓對的通過。

20 分鐘

查看預期成品 Demo ↗

## 學生現在完成

1. 名稱加入 Required 與 20 字限制。
2. Email 加入 Required 與 EmailAddress。
3. 場次設為必選。
4. 先檢查 ModelState，再顯示成功。

## 完成條件

- 空白：錯誤，沒有成功摘要。
- Email=abc：格式錯誤。
- 合法假資料：摘要完全一致。
- 修正錯誤後能再次送出。

**老師巡視：**要求學生先重現失敗，再改程式。

**第 10 分鐘：**尚未完成者先做好名稱與 Email 兩項驗證。

**快學生：**測試名稱第 20 與第 21 字，不增加欄位。

- [ ] 我已在 localhost 完成空白、錯誤格式、正確資料三條測試。

時間到：兩人互換測試，不直接替對方改程式

PAIR TEST

# 測試者不改程式，只交付一份可重現的回報。

1

### 輸入

清楚說明用了哪一組假資料。

2

### 操作

說明按了什麼、在哪個畫面發生。

3

### 結果

比較預期與實際，不只說「壞掉」。

回報句型：「我輸入____，按下送出後，預期____，實際看到____；請先檢查____。」

使用課堂假資料；不要輸入真實 Email、電話、密碼、連線字串或 API key。

FINAL BOSS · REPAIR WITH EVIDENCE

# 不是改一個字就過關，要重現、修正、測試、說明。

**基礎救援**
一個問題，有明確修改範圍

**標準任務**
自己定位，測成功與失敗

**進階委託**
加入小需求與邊界測試

**抽情境任務**

基礎救援　8–12 分鐘

## 幽靈報名成功

**情境**：所有欄位空白，畫面卻顯示報名成功。

**任務**：調整 OnPost，讓錯誤資料停在表單。

**限制**：保留 DataAnnotations；必須使用 ModelState.IsValid。

**必測**：空白失敗；合法資料成功。

**口頭說明**：為什麼要先檢查再設定 Submitted？

☐ 老師已看過我的重現、修改、測試與說明，確認通過。

同級抽題，避免運氣造成難度失衡

MISSION COMPLETE

# 你完成的不是一張表單，是第一條後端資料流程。

- [ ] 表單可操作
- [ ] 後端收到資料
- [ ] 錯誤資料被擋住
- [ ] Boss 有證據通過
- [ ] 能說明五個站
- [ ] 知道尚未存資料庫

**本週**
收到並驗證一筆資料

→

**下週**
把多筆報名排成清單

→

**再下一步**
寫入資料庫，重啟後仍存在

**完成標準**：不是程式碼很多，而是三條測試都能重現，而且你能解釋資料走過哪裡。